

Aim :- Decision Tree Learning

1. Implement the Decision Tree Learning algorithm to build a decision tree for the given diabetes dataset.
2. Evaluate the accuracy and effectiveness of the decision tree on test data.
3. Visualize and interpret the generated decision tree

Description :-

The program implements a Decision Tree Classification algorithm using the Diabetes dataset. The model learns patterns from the training data and predicts whether the outcome indicates No Diabetes or Diabetes.

Step	Code / Function	Description
1	<code>import pandas as pd</code>	Imports Pandas for reading and processing the dataset.
2	<code>import matplotlib.pyplot as plt</code>	Imports Matplotlib for displaying the decision tree.
3	<code>from sklearn.model_selection import train_test_split</code>	Splits the dataset into training and testing data.
4	<code>DecisionTreeClassifier</code>	Creates the Decision Tree classification model.
5	<code>accuracy_score</code>	Calculates the accuracy of the model.
6	<code>classification_report</code>	Displays precision, recall, and F1-score.
7	<code>confusion_matrix</code>	Shows correct and incorrect predictions.
8	<code>pd.read_csv("diabetes.csv")</code>	Loads the Diabetes dataset from the CSV file.
9	<code>df.drop("Outcome", axis=1)</code>	Removes the target column from the input features.

10	<code>X = df.drop("Outcome", axis=1)</code>	Stores the input features in X.
11	<code>y = df["Outcome"]</code>	Stores the target/output variable in y.
12	<code>train_test_split()</code>	Divides data into 80% training and 20% testing data.
13	<code>DecisionTreeClassifier(criterion="entropy")</code>	Creates the Decision Tree using entropy as the splitting criterion.
14	<code>model.fit(X_train, y_train)</code>	Trains the Decision Tree using training data.
15	<code>model.predict(X_test)</code>	Predicts the classes for the test data.
16	<code>accuracy_score()</code>	Calculates the prediction accuracy.
17	<code>classification_report()</code>	Evaluates precision, recall, and F1-score.
18	<code>confusion_matrix()</code>	Displays the number of correct and incorrect classifications.
19	<code>plot_tree()</code>	Generates a graphical representation of the trained Decision Tree.
20	<code>plt.show()</code>	Displays the Decision Tree visualization.

Visualization table :-The practical produces several useful outputs:

Visualization / Output	Purpose	What it Shows
Dataset Table	Understand the dataset	Features and Outcome values
Training/Test Split	Prepare data for ML	80% training and 20% testing
Accuracy	Measure overall performance	Percentage of correctly classified samples
Classification Report	Detailed evaluation	Precision, Recall and F1-score
Confusion Matrix	Analyze predictions	Correct and incorrect No Diabetes/Diabetes predictions
Decision Tree	Visualize the model	Features, conditions, samples and predicted classes

Dataset Columns :-

Feature	Description
Pregnancies	Number of pregnancies
Glucose	Glucose concentration
BloodPressure	Blood pressure measurement
SkinThickness	Skin fold thickness
Insulin	Insulin measurement
BMI	Body Mass Index
DiabetesPedigreeFunction	Diabetes pedigree function value
Age	Age of the person
Outcome	Target: 0 = No Diabetes, 1 = Diabetes

CODE:

```

T090_PRACTICAL3_Ai.py - C:\practical\AI\T090_PRACTICAL3_Ai.py (0.13.0)
File Edit Format Run Options Window Help

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

df = pd.read_csv("diabetes.csv")

print("First 5 rows:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

X = df.drop("Outcome", axis=1)
y = df["Outcome"]

print("\nNumber of Features:", X.shape[1])
print("Number of Samples:", X.shape[0])

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])

stratify=y
)

print("\nTraining Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])

model = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=4,
    random_state=42
)

model.fit(X_train, y_train)

print("\nDecision Tree trained successfully!")

y_pred = model.predict(X_test)

print("\nPredicted Values:")
print(y_pred)

accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=["No Diabetes", "Diabetes"]
    )
)

```

SHETH L.U.J. AND SIR M.V. COLLEGE

```
T090_PRACTICAL3_AI.py - C:/practical/AI/T090_PRACTICAL3_AI.py (3.13.0)
File Edit Format Run Options Window Help

print(y_pred)

accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=["No Diabetes", "Diabetes"]
    )
)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

plt.figure(figsize=(20, 10))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=["No Diabetes", "Diabetes"],
    filled=True,
    rounded=True
)

plt.title("Decision Tree - Diabetes Classification")
plt.show()

Activate Windows
Go to Settings to activate Windows.

Ln: 64 Col: 0
```

OUTPUT:

```
IDLE Shell 3.13.0
File Edit Shell Debug Options Window Help

Python 3.13.0 (tags/v3.13.0:60403a5, Oct 7 2024, 09:38:07) [MSC v.1941 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
===== RESTART: C:/practical/AI/T090_PRACTICAL3_AI.py =====
First 5 rows:
Pregnancies  Glucose  BloodPressure  ...  DiabetesPedigreeFunction  Age  Outcome
0           6      148           72.0  ...           0.627      50           1
1           1       85           66.0  ...           0.351      31           0
2           8      183           64.0  ...           0.672      32           1
3           1       89           66.0  ...           0.167      21           0
4           0      137           40.0  ...           2.288      33           1

[5 rows x 9 columns]

Dataset Shape:
(768, 9)

Number of Features: 8
Number of Samples: 768

Training Samples: 614
Testing Samples: 154

Decision Tree trained successfully!

Predicted Values:
[0 0 0 1 0 0 1 1 0 1 0 1 0 1 0 0 1 1 0 0 1 0 0 0 1 1 0 0 1 0 1 0 0 0 0 1 1 1 0
 0 1 0 0 0 0 1 0 1 1 0 1 1 0 1 1 0 0 0 1 0 1 0 1 1 0 1 0 0 1 0 0 0 1
 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 1 1 0 0 1 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0
 1 0 1 0 0 1 0 1 0 0 0 0 1 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 1 1
 0 0 0 0 1 0]
```

```

IDLE Shell 3.13.0"
File Edit Shell Debug Options Window Help
Number of Features: 8
Number of Samples: 768

Training Samples: 614
Testing Samples: 154

Decision Tree trained successfully!

Predicted Values:
[0 0 0 1 0 0 1 1 0 1 0 1 0 0 1 1 0 0 1 1 0 0 1 0 0 0 1 1 1 0 0 1 0 1 0 0 0 0 0 1 1 1 0
 0 1 0 0 0 0 1 0 1 1 1 0 1 1 1 0 1 1 0 0 0 1 0 1 0 1 1 0 1 0 0 1 0 0 1 0 0 0 1
 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 1 1 0 0 1 0 0 1 0 1 0 0 0 0 0 1 0 1 0 0 0 0
 1 0 1 0 0 1 0 1 0 0 0 0 1 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 1 1
 0 0 0 0 1 0]

Accuracy: 0.8571428571428571
Accuracy Percentage: 85.71428571428571 %

Classification Report:
              precision    recall  f1-score   support

No Diabetes      0.88        0.90        0.89        100
Diabetes         0.81        0.78        0.79         54

   accuracy          0.86
  macro avg          0.84
weighted avg          0.86

Confusion Matrix:
[[90 10]
 [12 42]]

Activate Windows
Go to Settings to activate Windows.

Ln: 40 Col: 0
2123
02-09-2024

```

[illegible]